Reporte

Equipo 16	
Nombre: Lesly Adamaris Cotlame Nieto Christopher Bethancourt Contreras Francisco Mora Ruiz	**Matrícula**: 2226958 2226948 2226912
Nombre del curso: Componentes de lógica de negocios.	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 4	**Actividad**: Laboratorio guiado. Programa para administrar pedidos de una tienda ficticia TechStore.
Fecha: 28/10/2026	

Bibliografia (Formato APA)

- Beck, K. (2002). Test-driven development: By example. Addison-Wesley Professional.
- Evans, E. (2003). Domain-driven design: Tackling complexity in the heart of software. Addison-Wesley.
- Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.
- Millett, S., & Tune, N. (2015). Patterns, principles, and practices of domain-driven design. Wrox.
- Vernon, V. (2013). Implementing domain-driven design. Addison-Wesley Professional.

Objetivo:

El estudiante implementará los componentes de lógica de negocio diseñados en U3: entidades DDD, reglas con Specification Pattern, máquinas de estado, Domain Events y APIs REST.

1. Diagrama de estados

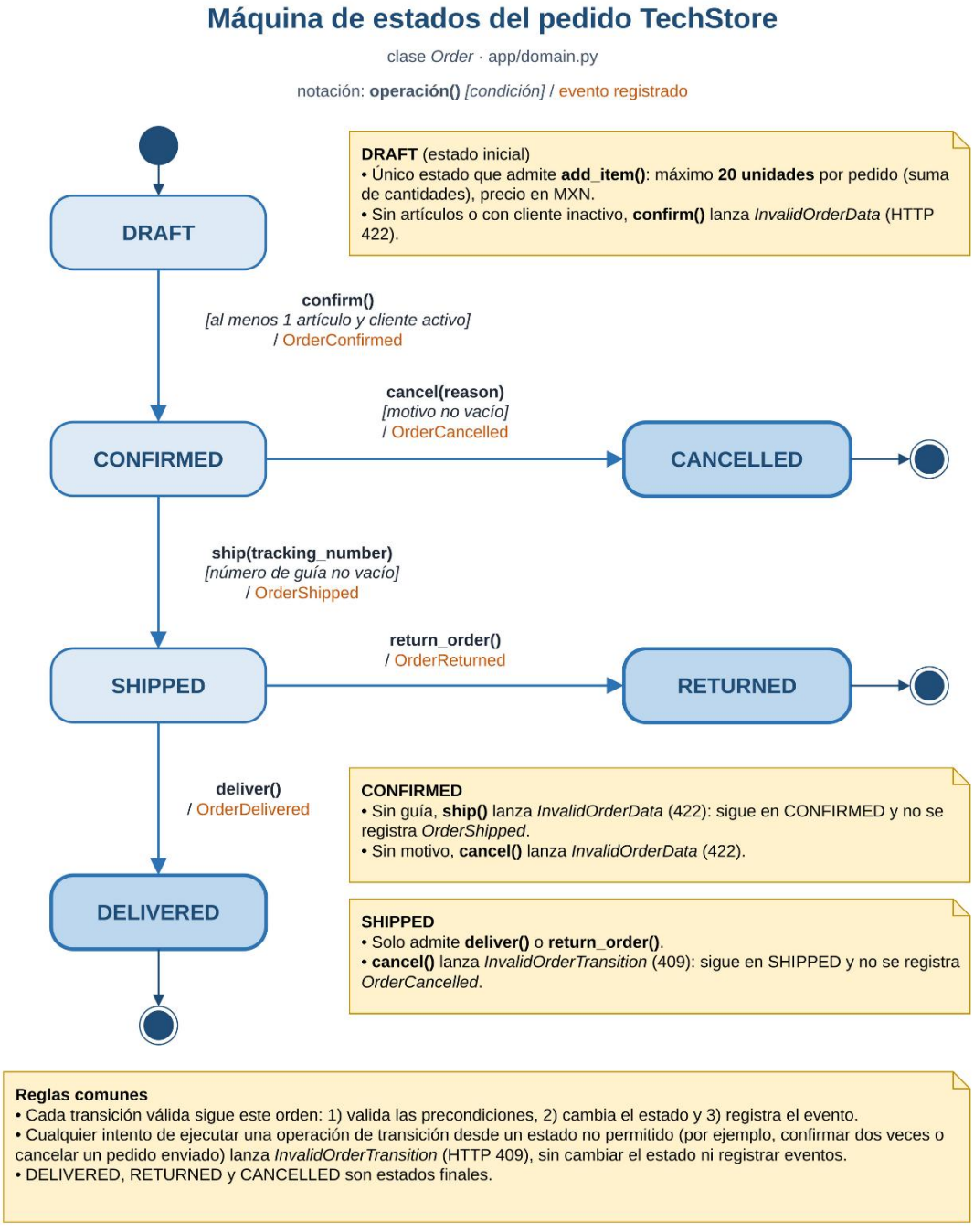


Figura 1. *Máquina de estados del pedido TechStore.*

El pedido nace en DRAFT. Solo existen cinco transiciones de negocio; DELIVERED, RETURNED y CANCELLED son estados finales. Cualquier intento de ejecutar una operación de transición desde un estado no permitido (por ejemplo, confirmar dos veces o cancelar un pedido enviado) lanza InvalidOrderTransition (HTTP 409), sin cambiar el estado ni registrar eventos.

2. Transiciones permitidas y precondiciones

Transición y método	Precondiciones (se validan en este orden)	Si falla	Evento si tiene éxito
DRAFT → CONFIRMED confirm()	1) estado DRAFT; 2) al menos un artículo; 3) cliente activo	1) InvalidOrderTransition (409) 2–3) InvalidOrderData (422)	OrderConfirmed
CONFIRMED → SHIPPED ship(tracking_number)	1) estado CONFIRMED; 2) número de guía no vacío	1) InvalidOrderTransition (409) 2) InvalidOrderData (422)	OrderShipped
CONFIRMED → CANCELLED cancel(reason)	1) estado CONFIRMED; 2) motivo no vacío	1) InvalidOrderTransition (409) 2) InvalidOrderData (422)	OrderCancelled
SHIPPED → DELIVERED deliver()	estado SHIPPED	InvalidOrderTransition (409)	OrderDelivered
SHIPPED → RETURNED return_order()	estado SHIPPED	InvalidOrderTransition (409)	OrderReturned

Ejemplos de transiciones prohibidas: DRAFT → SHIPPED (enviar sin confirmar), SHIPPED → CANCELLED (cancelar un pedido enviado, E13), CONFIRMED → CONFIRMED (confirmar dos veces, E12), CONFIRMED → DELIVERED (entregar sin enviar) y cualquier operación de transición desde DELIVERED, RETURNED o CANCELLED.

3. Estructura de un evento

Fragmento de app/domain.py (los comentarios se agregaron para explicar cada campo; _utc_now() devuelve datetime.now(timezone.utc)):

```
@dataclass(frozen=True)
class DomainEvent:
    name: str  # "OrderConfirmed", "OrderShipped", ...
    order_id: str  # pedido al que pertenece
    occurred_at: datetime = field(default_factory=_utc_now)  # fecha y hora en UTC
```

- Inmutable (frozen=True): una vez registrado, el hecho no se puede alterar.
- Fecha con zona horaria UTC: evita ambigüedades entre zonas horarias.
- Se guardan en Order.events, que se expone como tupla de solo lectura.

4. Los cinco eventos, uno por uno

4.1 OrderConfirmed

- Hecho que representa: el pedido quedó confirmado; el cliente ya no puede agregar artículos.
- Operación que lo genera: Order.confirm() (API: POST /orders/{order_id}/confirm).
- Condiciones para registrarlo: el pedido estaba en DRAFT, tenía al menos un artículo y el cliente estaba activo.
- Cuando no se registra: confirmar O007 (vacío) o O003 (cliente inactivo) lanza InvalidOrderData; una segunda confirmación de O013 lanza InvalidOrderTransition. En los tres casos el número de eventos no cambia.

4.2 OrderShipped

- Hecho que representa: el pedido salió del almacén con un número de guía.
- Operación que lo genera: Order.ship(tracking_number) (API: POST /orders/{order_id}/ship con {"tracking_number": "..."}).
- Condiciones para registrarlo: el pedido estaba en CONFIRMED y la guía no está vacía. La guía se guarda en tracking_number.
- Cuando no se registra: en E11, O012 confirmado intenta enviarse sin guía: InvalidOrderData, sigue en CONFIRMED y solo conserva OrderConfirmed.

4.3 OrderCancelled

- Hecho que representa: el pedido confirmado se canceló por un motivo registrado.
- Operación que lo genera: Order.cancel(reason) (API: POST /orders/{order_id}/cancel con {"reason": "..."}).
- Condiciones para registrarlo: el pedido estaba en CONFIRMED y el motivo no está vacío. El motivo se guarda en cancellation_reason.
- Cuando no se registra: en E13, O014 ya está en SHIPPED; cancel() lanza InvalidOrderTransition, el pedido sigue en SHIPPED y no aparece OrderCancelled. Un motivo vacío lanza InvalidOrderData.

4.4 OrderDelivered

- Hecho que representa: el cliente recibió el pedido.
- Operación que lo genera: Order.deliver() (API: POST /orders/{order_id}/deliver).
- Condiciones para registrarlo: el pedido estaba en SHIPPED.
- Ejemplo real: E15 (O016): confirmar → enviar → entregar; eventos finales OrderConfirmed, OrderShipped, OrderDelivered.

4.5 OrderReturned

- Hecho que representa: el pedido enviado fue devuelto.
- Operación que lo genera: Order.return_order() (API: POST /orders/{order_id}/return).
- Condiciones para registrarlo: el pedido estaba en SHIPPED.
- Ejemplo real: E14 (O015): confirmar → enviar → devolver; eventos finales OrderConfirmed, OrderShipped, OrderReturned.

5. Resumen: evento, transición y comportamiento ante un rechazo

Evento	Transición	Estado resultante	Si la operación se rechaza
OrderConfirmed	DRAFT → CONFIRMED	CONFIRMED	sigue en DRAFT (o en su estado actual); no se agrega evento
OrderShipped	CONFIRMED → SHIPPED	SHIPPED	sigue en CONFIRMED; no se guarda guía ni evento
OrderCancelled	CONFIRMED → CANCELLED	CANCELLED	conserva su estado (por ejemplo, SHIPPED); no se guarda motivo ni evento
OrderDelivered	SHIPPED → DELIVERED	DELIVERED	conserva su estado; no se agrega evento
OrderReturned	SHIPPED → RETURNED	RETURNED	conserva su estado; no se agrega evento

6. Momento en que se registran

Todos los métodos de transición siguen el mismo orden (práctica guiada, sección 9):

1. Validar las precondiciones. Si alguna falla, se lanza la excepción y el método termina.
2. Cambiar el estado (self._status = ...).
3. Registrar el evento (self._record("OrderConfirmed")).

Fragmento de Order.confirm() en app/domain.py (los mensajes de error se abrevian con «...» y los comentarios numerados se agregaron para la explicación):

```
def confirm(self) -> None:
    self._require_status(OrderStatus.DRAFT, "confirmar")  # 1. validar el estado
    if not self._items:  # 1. validar que tenga artículos
        raise InvalidOrderData(...)
    if not self._customer.active:  # 1. validar que el cliente esté activo
        raise InvalidOrderData(...)
    self._status = OrderStatus.CONFIRMED  # 2. cambiar el estado
    self._record(ORDER_CONFIRMED)  # 3. registrar el evento
```

La excepción se lanza en el paso 1, antes de los pasos 2 y 3. Por lo tanto, una operación fallida no cambia el estado, no agrega eventos y no modifica la guía, el motivo ni los artículos.

Caso	Operación fallida	Eventos al final
E11 (O012)	ship("") después de confirmar	solo OrderConfirmed; sigue CONFIRMED
E12 (O013)	segundo confirm()	un solo OrderConfirmed
E13 (O014)	cancel() desde SHIPPED	OrderConfirmed, OrderShipped; no hay OrderCancelled
E03 / E06	confirm() con cliente inactivo o pedido vacío	ninguno; sigue DRAFT

Pruebas que lo verifican: test_matriz_de_transiciones (30 combinaciones de estado por operación), test_confirm_dos_veces_no_duplica_evento, test_ship_sin_guia_se_rechaza_y_permanece_confirmed, test_cancel_desde_shipped_se_rechaza_sin_evento y test_se_generan_los_cinco_tipos_de_evento.

7. ¿Por qué solo después de una transición válida?

Un Domain Event indica que un hecho del negocio ya ocurrió. Por ejemplo, OrderShipped comunica que un pedido fue enviado. Si el evento se registrara antes de validar las condiciones, el sistema podría indicar que ocurrió una operación que en realidad fue rechazada. Por eso, el código primero valida las precondiciones, después cambia el estado del pedido y finalmente registra el evento. Si la operación falla, el estado no cambia y el evento correspondiente no se registra.